

1 Higgs

1.1 Why divide the total signal MC weights by 6?

- Original MC data simulated conditions for 10 fb^{-1} , whereas new real data corresponds to 36.1 fb^{-1} .
- The stricter selection criteria applied to the new dataset reduced the overall signal efficiency by about a factor of six.
- Dividing by six ensures the MC predictions match the absolute event yield observed in the new dataset.
- This adjustment affects only the scale of signal strength, not the statistical significance, which depends solely on relative likelihood fits.

1.2 Why choose higher photon p_T thresholds (50 GeV / 35 GeV)?

- With increased data volume, we can afford a slight efficiency reduction to significantly reduce electron and fake-photon contamination in the background.
- Genuine Higgs signal photons typically have much higher p_T , thus losing minimal signal events with these stricter cuts.

1.3 Why select a 4th-order Bernstein polynomial for background fitting?

- Bernstein polynomials effectively approximate any smooth distribution within a finite interval.
- Coefficients naturally remain in the range $[0,1]$, avoiding negative values.
- A 4th-order polynomial is sufficiently flexible to accurately model the background without overfitting.

1.4 Under Wilks' theorem, when can $-2\Delta \ln L$ be approximated by a χ^2 distribution?

- Wilks' theorem requires large samples, true parameter values away from boundaries, and smooth, regular likelihood functions.
- These conditions hold here: large sample size (10,000 events), and adding only one extra free parameter (signal strength μ).
- Thus, $-2\Delta \ln L$ safely approximates a χ^2 distribution with one degree of freedom.

1.5 Why fit background first with $\mu = 0$, then release μ for signal fitting?

- Fitting background first provides stable initial values for subsequent optimization.
- It prevents numerical issues such as convergence to local minima when simultaneously fitting multiple parameters.
- Clearly establishes the maximum likelihood under the background-only hypothesis for subsequent likelihood-ratio testing.

1.6 Why plot residuals, and what do they indicate?

- Residual plots (data minus background fit) clearly show the excess around 125 GeV, highlighting the signal structure.
- Residuals also detect systematic deviations or biases in the background model.

1.7 Why is the minimum of the μ profile-likelihood scan close to $\mu \approx 1$, and why does $\Delta \ln L = 1$ correspond to 1σ ?

- Signal strength parameter μ is defined relative to the Standard Model expectation; thus, perfect agreement with the Standard Model would yield $\mu \approx 1$.
- With only one free parameter, a change of $\Delta(-2 \ln L) = 1$ corresponds exactly to a 1σ deviation, and $\Delta(-2 \ln L) = 4$ to 2σ , due to properties of the χ^2 distribution.

1.8 Describe the complete fitting procedure (overall description without specific formulae).

- **Signal fit:** Fit the pure-MC signal peak separately to determine fixed parameters (peak position, widths, tail).
- **Background fit:** Fit data with the signal strength parameter μ fixed at zero, obtaining the maximum likelihood under a background-only hypothesis.
- **Combined fit:** Fit data again allowing signal strength μ to vary, thus determining the maximum likelihood under the signal-plus-background hypothesis.
- Calculate the statistical significance of the observed signal using the likelihood difference between the two fits.

2 Equivariance

2.1 How to implement a rotation-equivariant convolution layer?

- Rotate the input image into four orientations (0° , 90° , 180° , 270°).
- Apply the same convolutional kernel to each rotated version.
- Rotate each output feature map back to the original orientation.
- Concatenate these feature maps along the channel dimension, ensuring shared weights and rotation-equivariance.

2.2 Why does this implementation guarantee rotation equivariance?

- For each orientation (0° , 90° , 180° , 270°), the input is rotated, convolved with the **same shared kernel**, and then rotated back to the original reference frame.
- Thus, if the input is rotated, each output feature map will rotate accordingly. Due to separate storage of orientations, this manifests as a **cyclic permutation** of feature channels.
- Hence, this process explicitly ensures exact rotation equivariance.

2.3 What is the advantage of equivariant layers over standard CNNs with data augmentation?

- Standard CNNs with augmentation must learn orientation invariance implicitly, requiring more training data and resources.
- Equivariant layers explicitly share parameters across rotations, achieving better data efficiency and fewer parameters.

2.4 How would you experimentally verify equivariance of your convolutional layer?

- Rotate input MNIST images by 0° , 90° , 180° , and 270° .
- Pass each rotated image through the equivariant layer independently.
- Rotate outputs back to the original orientation and compare them element-wise.
- Equivariance is verified if the difference between outputs is negligible (within numerical precision).

2.5 Why do we prefer rotation-equivariance rather than rotation-invariance directly?

- Rotation-invariant layers immediately discard orientation information, limiting flexibility.
- Equivariant layers preserve orientation details, enabling downstream tasks such as pose estimation or segmentation.
- Final invariance, if needed, can easily be achieved through pooling at later stages, maintaining task flexibility.

3 XGBoost

3.1 What is the goal of the signal-background separation task (B2)?

- Identify signal events ($B_s^0 \rightarrow D_s^- \pi^+$) from similar background using machine learning.
- Maximize signal-to-background (S/B) ratio near the B_s^0 mass peak.

3.2 Where do the signal and background samples come from?

- **Signal:** Pure Monte-Carlo simulation of $B_s^0 \rightarrow D_s^- \pi^+$ events.
- **Background:** Real experimental data from high-mass region (> 5.6 GeV), assumed signal-free.

3.3 Which features were chosen for the BDT training and why?

- Kinematic and topological variables (flight distance, angles, transverse momenta, impact parameters).
- Explicitly exclude invariant mass to avoid bias (sculpting).

3.4 How was the BDT performance evaluated?

- Numerical metrics: accuracy, ROC-AUC, KS test (train-test consistency).
- Visualization: Probability histograms of signal/background predictions for visual confirmation.

3.5 How was the optimal cut determined?

- Initial loose manual cuts remove obvious background.
- Iterative optimization (grid or greedy search) maximizes the figure of merit ($FOM = S/\sqrt{S+B}$).
- Stop when no further significant improvement is found.

3.6 Why not include B_s mass directly as a feature?

- Including mass leads to artificial enhancement (sculpting), distorting the mass distribution.
- Proper analysis should rely only on mass-independent features.

3.7 Advantages and disadvantages of using a BDT compared to neural networks?

- **Pros:** Fast training, robust to feature scales, interpretable.
- **Cons:** Limited extrapolation.
- Neural networks excel with more data but require careful systematics handling.

3.8 How was overfitting avoided in this analysis?

- Data split into training/testing sets.
- Monitored distributions with KS test.
- Limited model complexity (max depth=3, learning rate=0.1).

4 Flavor Tagging

4.1 What is Thrust?

Thrust (T) quantifies how "jet-like" an e^+e^- event is, measuring momentum collimation. Mathematically:

$$T = \max_{\vec{n}} \frac{\sum_i |\vec{p}_i \cdot \vec{n}|}{\sum_i |\vec{p}_i|}, \quad \frac{1}{2} \leq T \leq 1$$

Here, $\vec{p}_i$ are particle momenta, and the direction $\vec{n}$ maximizing this ratio is the thrust axis.

- $T \approx 1$: back-to-back jets (highly collimated)
- $T \approx \frac{1}{2}$: isotropic distribution (spherical events)

4.2 Why are the jets back-to-back at the Z pole?

$$\begin{aligned} \sqrt{s} = m_Z \simeq 91.2 \text{ GeV}, \quad \vec{p}_{e^-} + \vec{p}_{e^+} &= \vec{0} \\ \Rightarrow \vec{p}_Z = \vec{0} &\implies Z^0 \rightarrow q + \bar{q} : \vec{p}_q + \vec{p}_{\bar{q}} = \vec{0}. \end{aligned}$$

Because the Z^0 is produced nearly at rest, momentum conservation forces the quark and antiquark (and hence the two jets) to have equal magnitudes and opposite directions.

Practical check in data

1. Determine the thrust axis $\vec{n}_T$ by maximising

$$T = \frac{\sum_i |\vec{p}_i \cdot \vec{n}|}{\sum_i |\vec{p}_i|}.$$

2. Collect particles within $\Delta\theta < 10^\circ$ around $+\vec{n}_T$ and $-\vec{n}_T$ and sum their momenta, $\vec{P}_1$ and $\vec{P}_2$.
3. Verify that

$$|\vec{P}_1 + \vec{P}_2| \approx 0 \quad (\text{e.g. } \frac{|\vec{P}_1 + \vec{P}_2|}{|\vec{P}_1| + |\vec{P}_2|} < 5\%),$$

which confirms the two jets are effectively back-to-back.

4.3 Why is the transverse momentum (p_T) important?

- **Boost-invariance:** In e^+e^- collisions, the beam direction defines the z -axis. The transverse momentum

$$p_T = \sqrt{p_x^2 + p_y^2}$$

is invariant under Lorentz boosts along the beam axis, making it ideal for comparing particles across events.

- **Flavor information:** Particles with highest p_T typically carry most of the jet energy and thus provide key flavor-discriminating information. Sorting particles by descending p_T maximizes relevant information within a fixed-length input.

4.4 Why prioritize charged particles as input features?

Charged particles leave clear tracks in the detector's tracker, allowing us to measure:

- Precise 3D vertex positions (to identify secondary vertices)
- Momentum $\vec{p}$ from track curvature
- Particle charge (sign of curvature)

Neutral particles (e.g., photons, neutrons) do not leave tracks, providing only coarse energy and directional information from calorimeters.

Thus, charged particles offer cleaner and richer information for distinguishing jet flavours.

4.5 If the Z^0 has a significant boost, are the jets still back-to-back? Why?

In the Z^0 rest frame, the two quarks have equal and opposite momenta:

$$\vec{p}_q = -\vec{p}_{\bar{q}}.$$

But in the lab frame, if the Z^0 boson is boosted:

- Momenta along the boost direction increase, causing jets to be closer together.
- Transverse momenta (perpendicular to the boost) remain unchanged.

Thus, the jets are no longer exactly back-to-back; instead, their opening angle becomes smaller as the boost increases.

4.6 Why does a boosted Z^0 make flavour classification harder?

If the Z^0 boson is boosted, flavour tagging becomes more difficult because:

- Jet energies and momenta become larger, widening the range of feature values and complicating the learning process.
- The input distributions shift between training (unboosted) and real (boosted) data, harming model generalization.

4.7 Why Transformer instead of other models?

- **Permutation equivariance:** Without positional encoding, self-attention is permutation equivariant, naturally matching the fact that particles and vertices have no intrinsic order.
- **Parallel efficiency:** Transformers process the entire sequence simultaneously (unlike RNNs), enabling faster training and inference.
- **Global interactions:** Self-attention directly captures global particle–particle and particle–vertex relationships, unlike CNNs that rely on local receptive fields.
- **Robust masking:** Attention masks precisely remove padding values (-999), preventing noisy inputs from affecting the model.

4.8 Why this specific model design?

- **Separate embedding:** Particle and vertex features have different physics; each gets mapped independently to the same embedding space.
- **Concatenate & self-attention:** Particles and vertices are combined into a single sequence to learn their mutual interactions efficiently.
- **Attention mask:** Ensures padding values are ignored during attention computation.
- **Dimensionality reduction:** A small convolutional layer and global pooling quickly summarize Transformer outputs into a fixed-length vector for final classification.

4.9 What exactly does the attention mask do in your network?

- Inputs are padded with -999 to keep a fixed length.
- The attention mask sets attention weights to zero for these padded entries.
- This ensures the model ignores padding, focusing only on real particles and vertices.

4.10 Why does removing Particle ID improve accuracy?

- Particle IDs are arbitrary integers (ranging from single to triple digits), with no physical numeric meaning.
- Feeding these integers directly into the network introduces large numerical ranges, causing unstable gradients during training.
- Removing Particle IDs ensures the model learns only from stable, physically meaningful features, improving generalization and accuracy.

4.11 Why choose exactly 35 particles and 5 vertices per event?

- On average, each event has ~ 20 charged particles and ~ 43 total (charged + neutral) particles.
- Retaining 35 particles ensures most charged particles plus some neutrals are captured, while keeping inputs manageable.
- Similarly, events have an average of ~ 3.4 vertices; retaining the top 5 comfortably covers typical vertex multiplicities.

4.12 Why use both log-transform and z-score normalization?

- Features like energy, p_T , and χ^2 span large numeric ranges.
- A log-transform compresses these wide ranges into manageable scales.
- Z-score normalization (subtract mean, divide by standard deviation) further ensures stable training and prevents gradient issues.

4.13 Why use gradient clipping?

- Transformers have many parameters and can produce large gradients.
- Clipping gradients (with norm $\|\vec{g}\| < 1$) prevents sudden jumps during training.
- This stabilizes training and avoids early-stage loss explosions.

4.14 How do you ensure the model isn't overfitting?

- Compare output distributions of training and test sets using a Kolmogorov–Smirnov test (confirming $p > 0.05$).
- Use early stopping if validation loss stops decreasing.
- Verify on pull distributions (checking that most pull values are within ± 2).

4.15 If real data provides only particle ID probabilities (PID weights), how would you modify the network?

- Replace integer particle IDs with probability vectors (e.g., $[P_\pi, P_K, P_p, \dots]$).
- Add a simple linear embedding to map these probabilities to the same embedding dimension as other features.
- Concatenate the embedded PID probabilities with momentum and vertex features before inputting to the Transformer.

4.16 Why specifically choose vertex mass for the ablation study?

- Vertex mass clearly distinguishes jet flavors (as we've seen in flavor-wise distributions).
- It's closely correlated with vertex track multiplicity, another important flavor indicator.
- Removing vertex mass (alone or combined with track multiplicity) checks if the model relies on physically meaningful features.
- Significant accuracy drop after removal (about 5%) confirms the model uses physically interpretable information, not hidden biases.

4.17 How would you use SHAP for model interpretability?

- Select 1–2k representative events.
- Compute SHAP values for each input feature.
- Visualize these values to see which features (e.g., vertex mass, track multiplicity) influence predictions most.
- Check if high-importance features match physical intuition, confirming model reliability.

4.18 Are energy, momentum, and mass redundant?

- Theoretically, if momentum and mass are known, energy is redundant (from $E^2 = p^2 c^2 + m^2 c^4$).
- Practically, since measurements of E , p , and m come from different detectors, their errors are not fully correlated.
- Providing measured energy directly simplifies the network’s learning task, avoiding numerical instability.

4.19 Potential improvements via SHAP analysis

- First, run SHAP analysis to rank feature importance globally.
- Select top-ranked (20) features; remove consistently low-ranked ones.
- Retrain the network with this refined set, reducing noise and improving focus on physically meaningful variables.

4.20 Definition of particle and vertex features

- **Particle features:** detector measurements of final-state particles (momenta, energy, angles, charge, and linked vertex ID).
- **Vertex features:** reconstructed points where several particle tracks converge, indicating decay points of heavier particles like B , D , or Λ hadrons.